Experiment No. 5

Feature Extraction and Image Analysis
using SIFT and HOG Descriptors

Aim

To implement Scale-Invariant Feature Transform (SIFT) and Histogram of Oriented Gradients (HOG) techniques for feature extraction and analyze their effectiveness in image representation, object recognition, and computer vision applications.

Purpose

Feature extraction is a fundamental step in computer vision that transforms raw image data into meaningful descriptors suitable for object recognition, image matching, tracking, and classification. Robust feature descriptors should be invariant to changes in scale, rotation, illumination, and viewpoint.

In this experiment, SIFT and HOG feature extraction are implemented on real-world images and compared in terms of feature detection, robustness, computational complexity, and suitability for different computer vision applications.

Prerequisites

- Basic knowledge of Digital Image Processing
- Familiarity with Python programming
- Python libraries: OpenCV (cv2), scikit-image, NumPy, and Matplotlib
- Understanding of image gradients, keypoints, and feature descriptors
- Jupyter Notebook, Google Colab, or Visual Studio Code

Steps (Procedure)

1. Import the required libraries and load a real-world image or dataset.
2. Convert the image to grayscale and perform basic preprocessing.
3. Apply SIFT to detect keypoints and compute feature descriptors.
4. Visualize the keypoints and analyze their distribution across the image.
5. Extract HOG features using suitable cell size, block size, and orientation bins.
6. Visualize the HOG descriptor to understand the gradient orientation patterns.
7. Compare SIFT and HOG with respect to scale invariance, rotation invariance, and computational efficiency.
8. Apply SIFT (or HOG) for basic image matching or object recognition using two similar images.
9. Evaluate the strengths and limitations of both techniques.
10. Document the observations and discuss suitable applications.

Key Concepts

- **SIFT pipeline:** (1) scale-space extrema detection using Difference of Gaussians (DoG), (2) keypoint localization and rejection of low-contrast and edge points, (3) orientation assignment from local gradient histograms, (4) 128-D descriptor from a 4 x 4 grid of 8-bin orientation histograms.

- **HOG pipeline:** (1) compute gradients Gx, Gy, (2) magnitude = sqrt(Gx^2 + Gy^2), angle = atan2(Gy, Gx), (3) build a 9-bin orientation histogram for each 8 x 8 cell, (4) normalize over overlapping 2 x 2 cell blocks, (5) concatenate into one feature vector.
- **HOG vector length** for a 128 x 256 image with the parameters used: 15 x 31 blocks x 4 cells x 9 bins = 16,740 values.
- **Matching:** nearest-neighbor search on descriptors, Lowe's ratio test (best distance < 0.75 x second best), then RANSAC to estimate a homography and keep only geometrically consistent inliers.

Observations

- SIFT finds keypoints at corners and blobs; each has a location, scale (size), and orientation, and a 128-dimensional descriptor. Keypoints cluster in textured regions and are rare in flat areas.
- HOG gives a dense, fixed-length descriptor over the whole image window; its visualization shows strong gradient orientations along object edges and outlines.
- SIFT matched an image to a rotated and scaled copy of itself with a high inlier ratio after RANSAC, confirming scale and rotation invariance.
- The HOG vector of a rotated image was very different (low cosine similarity), while a brightness change barely affected it, showing HOG is not rotation-invariant but is robust to illumination due to block normalization.
- SIFT is slower per image and produces variable numbers of keypoints, whereas HOG is faster and has a fixed length that suits classifiers such as SVM.

Source Code

File: experiment5_sift_hog.py

```
"""
Experiment 5: Feature Extraction and Image Analysis using SIFT and HOG
Install:  pip install opencv-python scikit-image numpy matplotlib
Usage:    python experiment5_sift_hog.py [image1] [image2]
          (image2 is optional; if omitted, a rotated + scaled copy of image1 is used)
"""
import os
import sys
import time
import cv2
import numpy as np
import matplotlib.pyplot as plt
from skimage.feature import hog
from skimage import exposure

os.makedirs("output", exist_ok=True)

# ---------------------------------------------------------------
# Step 1 & 2: Load image, convert to grayscale, light preprocessing
# ---------------------------------------------------------------
path1 = sys.argv[1] if len(sys.argv) > 1 else "input.jpg"
path2 = sys.argv[2] if len(sys.argv) > 2 else None

img1 = cv2.imread(path1)
if img1 is None:  # fallback: textured synthetic image (SIFT needs texture/corners)
    print(f"'{path1}' not found - generating a synthetic textured image.")
    rng = np.random.default_rng(1)
    img1 = np.full((400, 500, 3), 200, dtype=np.uint8)
    for _ in range(60):
        c = tuple(int(v) for v in rng.integers(0, 255, 3))
        p1 = (int(rng.integers(0, 500)), int(rng.integers(0, 400)))
        if rng.random() < 0.5:
            cv2.circle(img1, p1, int(rng.integers(8, 40)), c, -1)
        else:
            p2 = (p1[0] + int(rng.integers(10, 70)), p1[1] + int(rng.integers(10, 70)))
            cv2.rectangle(img1, p1, p2, c, -1)
    cv2.putText(img1, "SIFT HOG", (120, 220), cv2.FONT_HERSHEY_SIMPLEX, 2, (0, 0, 0), 4)

if path2 and cv2.imread(path2) is not None:
    img2 = cv2.imread(path2)
else:  # second image = rotated, scaled and brightness-changed version of the first
    h, w = img1.shape[:2]
    M = cv2.getRotationMatrix2D((w / 2, h / 2), 30, 0.7)
    img2 = cv2.warpAffine(img1, M, (w, h), borderValue=(200, 200, 200))
    img2 = cv2.convertScaleAbs(img2, alpha=0.9, beta=15)

gray1 = cv2.cvtColor(img1, cv2.COLOR_BGR2GRAY)
gray2 = cv2.cvtColor(img2, cv2.COLOR_BGR2GRAY)
gray1 = cv2.GaussianBlur(gray1, (3, 3), 0)       # mild denoising
gray2 = cv2.GaussianBlur(gray2, (3, 3), 0)

# ---------------------------------------------------------------
# Step 3 & 4: SIFT keypoints and descriptors
# ---------------------------------------------------------------
sift = cv2.SIFT_create(nfeatures=0, contrastThreshold=0.04, edgeThreshold=10)
t0 = time.perf_counter()
kp1, des1 = sift.detectAndCompute(gray1, None)
t_sift = (time.perf_counter() - t0) * 1000
kp2, des2 = sift.detectAndCompute(gray2, None)

print(f"SIFT: {len(kp1)} keypoints in image 1 ({t_sift:.1f} ms), {len(kp2)} in image 2")
print(f"SIFT descriptor shape: {des1.shape}  (each keypoint -> 128-D vector)")
k0 = kp1[0]
octave = k0.octave & 255
octave = octave - 256 if octave >= 128 else octave      # decode signed octave index
print(f"Example keypoint: pt={tuple(round(v, 1) for v in k0.pt)}, size(scale)={k0.size:.1f}, "
      f"angle={k0.angle:.1f} deg, response={k0.response:.4f}, octave={octave}")

vis_plain = cv2.drawKeypoints(img1, kp1, None, color=(0, 255, 0))
vis_rich = cv2.drawKeypoints(img1, kp1, None, color=(0, 255, 0),
                             flags=cv2.DRAW_MATCHES_FLAGS_DRAW_RICH_KEYPOINTS)  # scale + orientation

# Distribution of keypoints across the image (2D histogram)
xs = np.array([k.pt[0] for k in kp1])
ys = np.array([k.pt[1] for k in kp1])
sizes = np.array([k.size for k in kp1])

# ---------------------------------------------------------------
# Step 5 & 6: HOG features and visualisation
```

```
# ------------------------------------------------------------
hog_img = cv2.resize(gray1, (128, 256))                # fixed size, as in the HOG pedestrian detector
t0 = time.perf_counter()
hog_vec, hog_vis = hog(hog_img,
                       orientations=9,                  # orientation bins over 0-180 deg
                       pixels_per_cell=(8, 8),          # cell size
                       cells_per_block=(2, 2),          # block size (for normalisation)
                       block_norm="L2-Hys",
                       visualize=True)
t_hog = (time.perf_counter() - t0) * 1000
hog_vis = exposure.rescale_intensity(hog_vis, in_range=(0, hog_vis.max() + 1e-9))

cells = (256 // 8, 128 // 8)
blocks = (cells[0] - 1, cells[1] - 1)
print(f"\nHOG: image 128x256, cells {cells[1]}x{cells[0]}, blocks {blocks[1]}x{blocks[0]}")
print(f"HOG feature vector length = {blocks[0]*blocks[1]} blocks x 4 cells x 9 bins = {hog_vec.size}")
print(f"HOG computation time: {t_hog:.1f} ms")

# ------------------------------------------------------------
# Step 7: Invariance test - how similar is the HOG vector after rotation/scale?
# ------------------------------------------------------------
def hog_vector(g):
    return hog(cv2.resize(g, (128, 256)), orientations=9, pixels_per_cell=(8, 8),
               cells_per_block=(2, 2), block_norm="L2-Hys")

def cosine(a, b):
    return float(np.dot(a, b) / (np.linalg.norm(a) * np.linalg.norm(b) + 1e-12))

print("\n---- HOG similarity (cosine) between original and transformed image ----")
print(f"Image 2 (rotated/scaled)     : {cosine(hog_vec, hog_vector(gray2)):.3f}")
print(f"Brightness change only       : {cosine(hog_vec, hog_vector(cv2.convertScaleAbs(gray1, alpha=1, beta=40))):.3f}")
print(f"Rotated 30 deg only          : {cosine(hog_vec, hog_vector(cv2.warpAffine(gray1, cv2.getRotationMatrix2D((gray1.shape[1]/2,

# ------------------------------------------------------------
# Step 8: Image matching with SIFT (Lowe's ratio test + RANSAC)
# ------------------------------------------------------------
bf = cv2.BFMatcher(cv2.NORM_L2)
knn = bf.knnMatch(des1, des2, k=2)
good = [m for m, n in (p for p in knn if len(p) == 2) if m.distance < 0.75 * n.distance]
print(f"\nSIFT matching: {len(good)} good matches (ratio test 0.75) out of {len(knn)} candidates")

result = None
if len(good) >= 4:
    src = np.float32([kp1[m.queryIdx].pt for m in good]).reshape(-1, 1, 2)
    dst = np.float32([kp2[m.trainIdx].pt for m in good]).reshape(-1, 1, 2)
    H, inlier_mask = cv2.findHomography(src, dst, cv2.RANSAC, 5.0)
    inliers = int(inlier_mask.sum())
    print(f"RANSAC inliers: {inliers}/{len(good)}  ({100 * inliers / len(good):.1f}%)")
    h, w = gray1.shape
    box = np.float32([[0, 0], [w, 0], [w, h], [0, h]]).reshape(-1, 1, 2)
    img2_boxed = img2.copy()
    cv2.polylines(img2_boxed, [np.int32(cv2.perspectiveTransform(box, H))], True, (0, 0, 255), 3)
    result = cv2.drawMatches(img1, kp1, img2_boxed, kp2, good, None,
                             matchesMask=inlier_mask.ravel().tolist(),
                             flags=cv2.DrawMatchesFlags_NOT_DRAW_SINGLE_POINTS)
else:
    print("Not enough matches to estimate a homography.")

# ------------------------------------------------------------
# Display
# ------------------------------------------------------------
rgb = lambda x: cv2.cvtColor(x, cv2.COLOR_BGR2RGB)

fig, ax = plt.subplots(1, 3, figsize=(15, 4.5))
ax[0].imshow(rgb(img1)); ax[0].set_title("Original")
ax[1].imshow(rgb(vis_rich)); ax[1].set_title(f"SIFT keypoints ({len(kp1)}) - size = scale, line = orientation")
ax[2].hist2d(xs, ys, bins=(10, 8), range=[[0, gray1.shape[1]], [0, gray1.shape[0]]], cmap="viridis")
ax[2].invert_yaxis(); ax[2].set_title("Keypoint distribution")
for a in ax[:2]: a.axis("off")
plt.tight_layout(); plt.savefig("output/exp5_sift_keypoints.png", dpi=150); plt.show()

fig, ax = plt.subplots(1, 3, figsize=(12, 6))
ax[0].imshow(hog_img, cmap="gray"); ax[0].set_title("Input (128x256)")
ax[1].imshow(hog_vis, cmap="gray"); ax[1].set_title("HOG visualisation")
ax[2].hist(sizes, bins=20, color="steelblue"); ax[2].set_title("SIFT keypoint scale distribution")
for a in ax[:2]: a.axis("off")
plt.tight_layout(); plt.savefig("output/exp5_hog.png", dpi=150); plt.show()

if result is not None:
    plt.figure(figsize=(14, 6)); plt.imshow(rgb(result))
    plt.title(f"SIFT matching with ratio test + RANSAC ({inliers} inliers)"); plt.axis("off")
    plt.tight_layout(); plt.savefig("output/exp5_matches.png", dpi=150); plt.show()
```

Questions and Answers

1. What is feature extraction, and why is it important in computer vision?
Feature extraction transforms raw pixel data into a compact set of numerical descriptors (edges, corners, textures, gradients, shapes) that capture the meaningful content of an image. It is important because it:

- Reduces dimensionality and computation compared with raw pixels
- Highlights discriminative information and suppresses irrelevant variation
- Gives representations that are robust to scale, rotation, illumination, and noise
- Forms the input for classification, matching, detection, tracking, and retrieval

2. Explain the working principle of Scale-Invariant Feature Transform (SIFT).
SIFT detects and describes local features that are stable under scale and rotation changes. Its four main stages are:

1. **Scale-space extrema detection:** the image is blurred at many scales and Difference-of-Gaussian images are computed; pixels that are maxima or minima among their 26 neighbors (3 x 3 x 3 across scales) are candidate keypoints.
2. **Keypoint localization:** candidates are refined to sub-pixel accuracy, and low-contrast points and points on edges are discarded.
3. **Orientation assignment:** a histogram of local gradient directions is built and the dominant direction is assigned to the keypoint, giving rotation invariance.
4. **Descriptor generation:** the region around the keypoint (rotated to the dominant orientation and scaled) is divided into a 4 x 4 grid, and an 8-bin gradient histogram per cell gives a 4 x 4 x 8 = 128-D vector, which is normalized for illumination robustness.

3. What are keypoints and feature descriptors in image analysis?
A **keypoint** (interest point) is a distinctive, repeatable location in an image, such as a corner or blob, that can be found again under changes in viewpoint, scale, or lighting. It is described by its position, scale, orientation, and response strength. A **feature descriptor** is a numerical vector that summarizes the appearance of the neighborhood around a keypoint (for example, SIFT's 128-D vector, or ORB's binary string), so that keypoints from different images can be compared and matched using a distance measure.

4. Explain the concept of Histogram of Oriented Gradients (HOG) and its significance.
HOG describes the shape and appearance of an object through the distribution of local gradient directions. The image window is divided into small cells (e.g., 8 x 8 pixels); in each cell a histogram of gradient orientations (e.g., 9 bins over 0-180 degrees), weighted by gradient magnitude, is built. Cells are grouped into overlapping blocks (e.g., 2 x 2 cells), and each block is contrast-normalized. All block histograms are concatenated into the final feature vector.

- Captures edge and shape structure while ignoring color and absolute brightness
- Block normalization gives robustness to illumination and contrast changes
- Dense, fixed-length output works directly with SVM and other classifiers
- Famous for pedestrian detection (Dalal and Triggs, 2005)

5. Compare SIFT and HOG based on robustness, computational complexity, and practical applications.

Aspect	SIFT	HOG
Type	Sparse local keypoint detector + descriptor	Dense descriptor over a fixed window
Scale invariance	Yes (scale-space)	No (needs image pyramid or fixed size)
Rotation invariance	Yes (dominant orientation)	No (orientation-sensitive)

Aspect	SIFT	HOG
Illumination	Robust (normalized descriptor)	Robust (block normalization)
Output	Variable number of 128-D vectors	One fixed-length vector
Complexity	Higher (scale-space, many keypoints, matching cost)	Lower and fast, easy to compute densely
Applications	Image matching, stitching, 3D reconstruction, object recognition, SLAM	Pedestrian, face, and object detection with SVM, human pose and gesture recognition

6. Why is SIFT considered invariant to scale and rotation?

- **Scale:** keypoints are detected as extrema in a Gaussian scale-space, so each keypoint comes with the scale at which it is strongest. The descriptor is computed over a region proportional to that scale, so the same structure gives the same descriptor at different image sizes.
- **Rotation:** each keypoint is assigned a dominant gradient orientation, and the descriptor is computed relative to this orientation, so rotating the image rotates the reference direction and the descriptor stays the same.
- It is also partially robust to illumination change (descriptor normalization) and small viewpoint change.

7. Mention three real-world applications where HOG descriptors are commonly used.

1. Pedestrian detection in autonomous driving and driver-assistance systems
2. Face and facial landmark detection (e.g., dlib's HOG-based face detector)
3. Human action and gesture recognition, and vehicle and traffic sign detection in surveillance
4. (Also: object detection in robotics and text and handwriting recognition)

8. Why is feature extraction performed before image classification or object detection?

Raw images are very high-dimensional and contain redundant, noisy, and lighting-dependent information, so classifiers working on raw pixels need huge data and computation and generalize poorly. Feature extraction converts the image into a compact, informative, and more invariant representation, which makes classes easier to separate, reduces training time and overfitting, and improves accuracy. (In deep learning, convolutional layers learn this extraction automatically.)

9. What are the advantages and limitations of handcrafted feature descriptors compared to deep learning-based feature extraction?

Aspect	Handcrafted (SIFT, HOG, ORB)	Deep learning (CNN features)
Advantages	No training data or GPU needed; interpretable; fast to prototype; works well on small datasets; well-understood invariances	Learned automatically; highly discriminative; state-of-the-art accuracy; adapts to the task and domain; handles complex variation
Limitations	Designed manually, limited expressive power; less accurate on complex tasks; sensitive to parameters; weak against large appearance or viewpoint change	Needs large labeled datasets and heavy computation; less interpretable; can overfit or fail on out-of-distribution data

10. How do feature extraction techniques contribute to image matching, face recognition, and object detection systems?

- **Image matching:** invariant keypoints and descriptors (SIFT, ORB) are matched between images to align them, stitch panoramas, track motion, and reconstruct 3D structure.
- **Face recognition:** features (HOG, LBP, or deep embeddings) represent a face compactly and robustly to lighting and pose, and are compared by distance or fed to a classifier.
- **Object detection:** a feature representation (HOG with a sliding window and SVM, or CNN feature maps) is computed over regions and classified to find and localize objects.
- In all cases, good features reduce data size, increase robustness, and improve accuracy and speed.

Conclusion

SIFT and HOG feature extraction were implemented using OpenCV and scikit-image. SIFT detects scale and rotation invariant keypoints and was used to match an image with a rotated and scaled copy using the ratio test and RANSAC. HOG gives a dense, fixed-length, illumination-robust shape descriptor that is well suited to classifiers, but is sensitive to rotation and scale. SIFT is preferred for matching and registration tasks, and HOG for window-based detection and classification.